



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Luis Eduardo Vieira de Oliveira

Francisco Kauan Pereira Cavalcante

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclométrica por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclométrica por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclométrica por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclométrica por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13

5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Nome Projeto: AWS SAM Transform

O AWS SAM (Serverless Application Model) Transform é uma macro do AWS CloudFormation que atua na transformação de templates simplificados do SAM em templates completos e detalhados do CloudFormation. As principais capacidades incluem:

- **Extensão da sintaxe do CloudFormation:** Permite definir recursos serverless (como funções Lambda e APIs) de forma muito mais curta e simples;
- **Melhores práticas embutidas:** Configura automaticamente padrões seguros e recomendados pela AWS (como a criação automática de permissões e papéis IAM necessários);
- **Suporte a testes locais:** Integração com o AWS SAM CLI para permitir a depuração e o teste das aplicações diretamente na máquina do desenvolvedor antes do deploy.

O propósito do AWS SAM Transform é simplificar drasticamente a infraestrutura como código (IaC) para arquiteturas serverless na AWS. O projeto resolve um problema prático: escrever templates puros do CloudFormation para recursos como AWS Lambda exige dezenas de linhas de configuração repetitiva e complexa de segurança (IAM) e políticas de execução. Ao abstrair essa complexidade por trás de uma macro ("under the hood"), o SAM Transform reduz o tamanho dos arquivos de configuração, diminui a chance de erros manuais e acelera o ciclo de desenvolvimento de aplicações serverless.

2. Métricas Antes da Refatoração

Nesta seção você vai apresentar as principais métricas obtidas antes da refatoração para cada ferramenta, não precisa ser todas as métricas. Confira um exemplo abaixo:

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
samtranslator/model/sam_resources.py	90	4.92	19	443	C	to_cloudformation
samtranslator/model/api/api_generator.py	40	6.92	27	277	D	_construct_api_domain
samtranslator/model/eventsources/push.py	41	5.83	32	239	E	_add_swagger_integration
samtranslator/swagger/swagger.py	41	5.80	16	238	C	_add_vpc_resource_policy_for_method
samtranslator/utis/py27hash_fix.py	46	3.54	23	163	D	to_py27_compatible_template

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
samtranslator/internal/managed_policies.py	1	1.0	1	1	A	get_bundled_managed_policy_map
samtranslator/model/naming.py	2	1.0	1	2	A	implicit_api
samtranslator/model/sqs.py	2	1.0	1	2	A	sns_topic_send_message_role_policy
samtranslator/plugins/exceptions.py	2	1.0	1	2	A	__init__
samtranslator/translator/transform.py	2	1.0	1	2	A	transform

2.1.2 Complexidade Ciclômática por Função Antes da Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
samtranslator/model/eventsources/push.py	method	_add_swagger_integration	E	32
samtranslator/model/api/api_generator.py	method	_construct_api_domain	D	27
samtranslator/utis/py27hash_fix.py	function	to_py27_compatible_template	D	23
samtranslator/utis/py27hash_fix.py	function	_template_has_httpapi_resource_with_default_authorizer	D	23
samtranslator/model/api/api_generator.py	method	_openapi_postprocess	D	21

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
samtranslator/yaml_helper.py	function	yaml_parse	A	1
samtranslator/feature_toggle/dialup.py	method	__init__	A	1
samtranslator/feature_toggle/dialup.py	method	is_enabled	A	1
samtranslator/feature_toggle/dialup.py	method	__str__	A	1
samtranslator/feature_toggle/dialup.py	method	__init__ (DisabledDialup)	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
samtranslator/model/sam_resources.py	0.0	C
samtranslator/model/api/api_generator.py	0.0	C
samtranslator/model/eventsources/push.py	3.4246	C
samtranslator/swagger/swagger.py	11.1651	B
samtranslator/model/eventsources/pull.py	16.2651	B

Tabela de melhores métricas

arquivo	mi	rank_mi
samtranslator/validator/sam_schema/definitions/__init__.py	100.0	A
samtranslator/validator/sam_schema/__init__.py	100.0	A
samtranslator/validator/__init__.py	100.0	A
samtranslator/utis/types.py	100.0	A
samtranslator/utis/traverse.py	100.0	A

2.1.4 Métricas brutas Antes da Refatoração

loc total	lloc total	sloc total	comments total	multi
27358	14534	17170	1924	4098

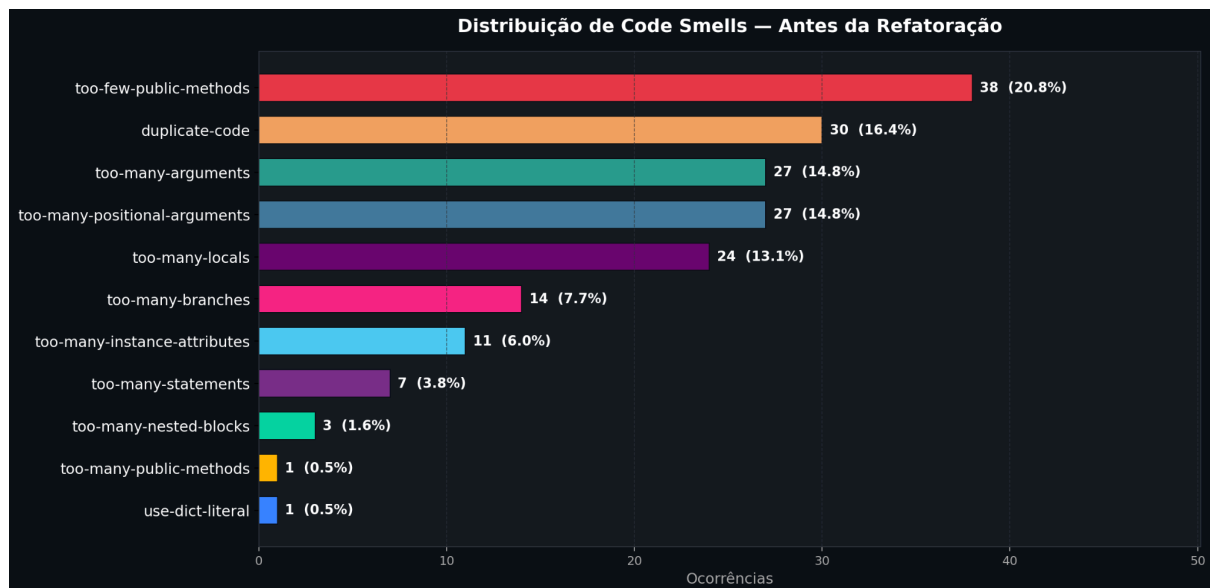
2.2 CodeCarbon Antes da Refatoração

Foi desenvolvido um script de carga para o projeto open-source [serverless-application-model](#) (AWS SAM), utilizado para gerar uma execução real e representativa a ser medida pelo CodeCarbon. O script executa a suíte de testes automatizados do módulo [samtranslator](#), responsável por traduzir templates SAM em templates CloudFormation, cobrindo cenários reais como funções Lambda, APIs, políticas de IAM e deployment preferences. Essa abordagem foi escolhida por refletir uma carga de trabalho diversificada e estatisticamente significativa, em vez de uma chamada isolada e repetida a uma única função, o que tornaria a medição pouco representativa do comportamento real do sistema.

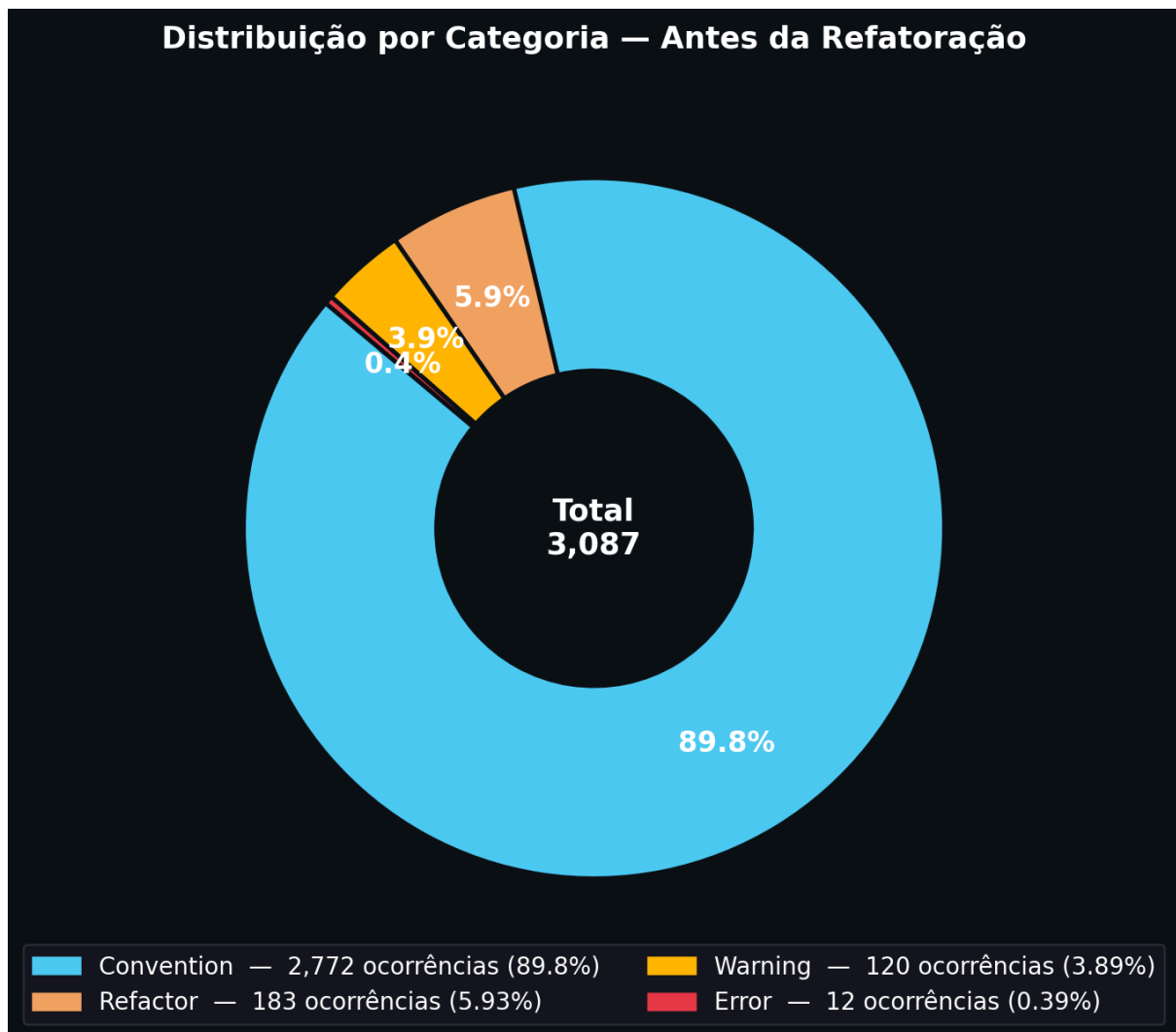
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
369.3715	0.00038511891896891154	1.0426328553459287 × 10 ⁻⁶	0.0006245139433800125	0.0005186216911087589	0.0039158795193487556

2.3 Pylint Antes da Refatoração

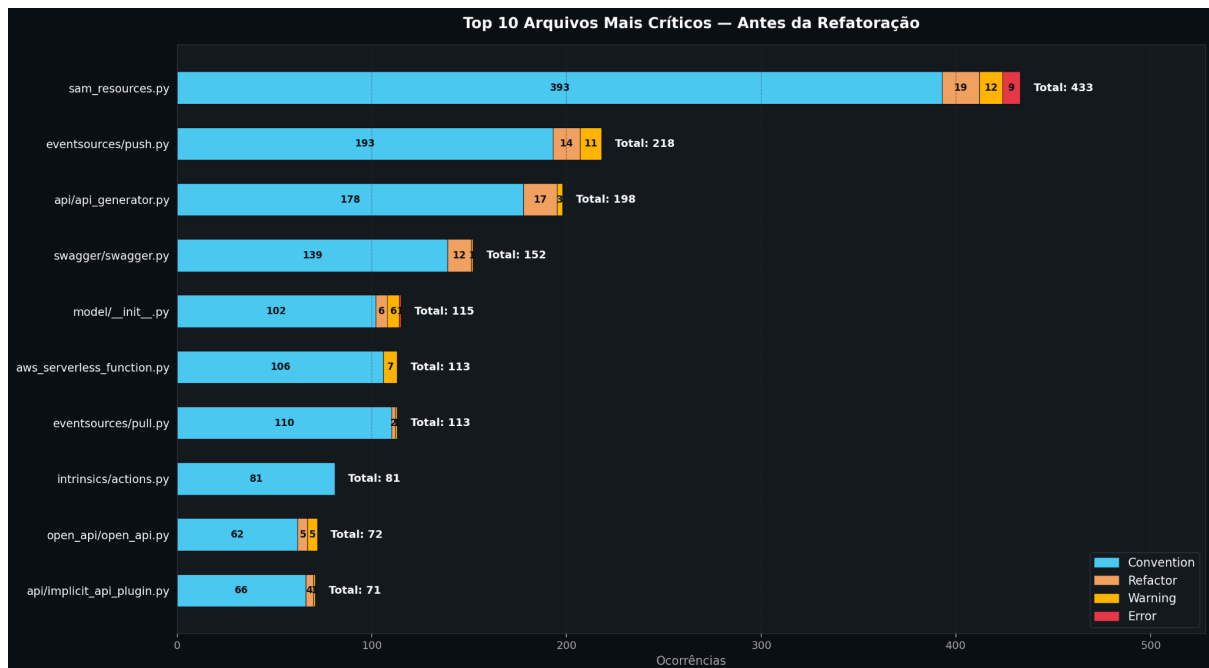
2.3.1 Distribuição do Smells Antes da Refatoração



2.3.2 Distribuição dos Problemas de Código Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração



2.3.4 Score Projeto Antes da Refatoração

Score: 7.3/10

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

A cobertura de testes é apresentada agrupada por diretório do módulo *samtranslator*, uma vez que a granularidade por arquivo resultaria em mais de 80 entradas, dificultando a visualização. O agrupamento por diretório permite identificar de forma mais clara quais subsistemas possuem

menor cobertura e demandam maior atenção durante a refatoração.

Code Coverage Report — Antes da Refatoração (samtranslator)				
Diretório	Statements	Missing	Branches	Cobertura
validator	127	72	40	43%
public	24	9	0	62%
third_party	61	7	22	84%
(raiz)	47	6	14	87%
open_api	359	27	170	90%
model	6589	267	2326	94%
swagger	633	26	294	94%
utils	401	19	200	94%
plugins	817	33	272	95%
translator	345	9	102	96%
feature_toggle	117	1	10	99%
intrinsic	262	3	98	99%
metrics	134	0	20	99%
internal	190	0	4	100%
parser	30	0	8	100%
policy_template_processor	96	0	16	100%
policy_templates_data	4	0	0	100%
sdk	96	0	32	100%
Total	10332	479	3628	95%

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Quantidade de testes que passaram: 4256

Quantidade de testes que falharam: 283

3. Processo de Refatoração

Modelo utilizado: DeepSeek V4 Flash Free

3.1 too-many-statements

Quantidade de ocorrências: 7

Tabela de Identificação:

module	obj	message
samtranslator.model.sam_resources	SamFunction.to_cloudformation	Too many statements (65/50)
samtranslator.model.api.apiv2_generator	ApiV2Generator._construct_api_domain	Too many statements (55/50)
samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain	Too many statements (73/50)
samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain_v2	Too many statements (53/50)
samtranslator.model.eventsources.pull	PullEventSource.to_cloudformation	Too many statements (77/50)
samtranslator.model.eventsources.push	Api._add_swagger_integration	Too many statements (72/50)
samtranslator.translator.translator	Translator.translate	Too many statements (78/50)

3.1.1 Ocorrência Número 1

module	obj	message
samtranslator.model.sam_resources	SamFunction.to_cloudformation	Too many statements (65/50)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/ZuBQoaWM>

Print consumo de token:

Revisão	Contexto ×	📄 sam_resources.py ×	+
Sessão	Mensagens		
Refatorar to_cloudformation (65 stmts)	32		
Provedor	Modelo		
OpenCode Zen	DeepSeek V4 Flash Free		
Limite de Contexto	Total de Tokens		
200.000	42.227		
Uso	Tokens de Entrada		
21%	234		
Tokens de Saída	Tokens de Raciocínio		
265	0		
Tokens de Cache (leitura/escrita)	Mensagens de Usuário		
41.728 / 0	2		
Mensagens do Assistente	Custo Total		
30	US\$ 0,00		
Sessão Criada	Última Atividade		
29 de jun. de 2026, 18:48	29 de jun. de 2026, 18:52		

Análise da refatoração: A análise do resultado indica uma melhoria significativa na coesão do código e na legibilidade, aplicando com sucesso o princípio de responsabilidade única (SRP). Centralizar a lógica complexa de provisionamento de alias e a criação condicional de roles IAM em métodos dedicados reduz o cansaço cognitivo ao ler `to_cloudformation`, facilitando manutenções futuras e a criação de testes unitários direcionados.

3.1.2 Ocorrência Número 2

module	obj	message
samtranslator.model.api.apiv2_generator	ApiV2Generator._construct_api_domain	Too many statements (55/50)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/TIWfgt5J>

Print consumo de token:

Revisão	Contexto ×	apiv2_generator.py ×	+
Sessão	Mensagens		
Refatorar _construct_api_domain	19		
Provedor	Modelo		
OpenCode Zen	DeepSeek V4 Flash Free		
Limite de Contexto	Total de Tokens		
200.000	40.949		
Uso	Tokens de Entrada		
20%	411		
Tokens de Saída	Tokens de Raciocínio		
90	0		
Tokens de Cache (leitura/escrita)	Mensagens de Usuário		
40.448 / 0	2		
Mensagens do Assistente	Custo Total		
17	US\$ 0,00		
Sessão Criada	Última Atividade		
29 de jun. de 2026, 19:45	29 de jun. de 2026, 19:50		

Análise da refatoração: A análise do resultado evidencia uma excelente aplicação dos princípios de Clean Code, transformando um método longo e propício a bugs em uma estrutura legível e de fácil manutenção. Ao isolar lógicas complexas como `MutualTlsAuthentication` e `EndpointConfiguration`, o código ganha em reuso e testabilidade isolada. O fato de os testes passarem sem alterações valida que o comportamento original do OpenCode foi preservado com sucesso.

3.1.3 Ocorrência Número 3

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain	Too many statements (73/50)

Seguiu o plano de refatoração: SIm.

Link da publicação OpenCode: <https://opncd.ai/share/axoHSW5y>

Print consumo de token:

Sessão	Mensagens
Refatorar R0915 (too-many-statements)	35
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	44.289
Uso	Tokens de Entrada
22%	293
Tokens de Saída	Tokens de Raciocínio
348	0
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
43.648 / 0	2
Mensagens do Assistente	Custo Total
33	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 20:08	29 de jun. de 2026, 20:13

Análise da refatoração: A análise do resultado demonstra um ganho expressivo em manutenibilidade e no combate à duplicação de código (DRY), especialmente pela eliminação do parsing inline que repetia a lógica de `_get_basepaths`. A separação clara entre a configuração do domínio e a geração de mapeamentos de caminhos isolou responsabilidades de níveis de abstração diferentes. A validação final por meio dos 39 testes garante que o comportamento do tradutor do SAM permaneceu intacto.

3.1.4 Ocorrência Número 4

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain_v2	Too many statements (53/50)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/79yBpIYL>

Print consumo de token:

Sessão	Mensagens
Refatorar_construct_api_domain_v2	30
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	57.266
Uso	Tokens de Entrada
29%	327
Tokens de Saída	Tokens de Raciocínio
225	10
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
56.704 / 0	2
Mensagens do Assistente	Custo Total
28	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 20:26	29 de jun. de 2026, 20:30

Análise da refatoração: A análise do resultado consolida o ganho de simetria e consistência arquitetural entre o motor de APIs REST (V1) e HTTP (V2) dentro do gerador do SAM. O uso explícito do sufixo **_v2** nas assinaturas foi uma decisão acertada de design, pois reflete o acoplamento correto com os tipos de recursos do CloudFormation específicos da V2 (**ApiGatewayDomainNameV2**), mantendo o código previsível. A preservação do comportamento de *early return* na integração com o Route53 reforça o isolamento seguro de efeitos colaterais.

3.1.5 Ocorrência Número 5

module	obj	message
--------	-----	---------

samtranslator.model.eventsour ces.pull	PullEventSource.to_cloudformation	Too many statements (77/50)
---	-----------------------------------	-----------------------------

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/7fcG4j6e>

Print consumo de token:

Sessão	Mensagens
Refatoração de PullEventSource.to_cloudformation	21
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	42.238
Uso	Tokens de Entrada
21%	220
Tokens de Saída	Tokens de Raciocínio
151	11
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
41.856 / 0	2
Mensagens do Assistente	Custo Total
19	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 20:33	29 de jun. de 2026, 20:37

Análise da refatoração: A análise do resultado evidencia um ganho expressivo de maturidade no design do código, que agora espelha as boas práticas de encapsulamento já iniciadas em outras partes do ecossistema do SAM. Ao isolar a complexidade das configurações de destino e do ecossistema Kafka em subrotinas dedicadas, o fluxo principal

tornou-se puramente declarativo e legível. O fato de todos os 48 testes passarem sem modificações comprova que o comportamento polimórfico dos mapeamentos de eventos foi preservado intacto.

3.1.6 Ocorrência Número 6

module	obj	message
samtranslator.model.eventsources.push	Api._add_swagger_integration	Too many statements (72/50)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/GrXk8Sqe>

Print consumo de token:

Sessão	Mensagens
Refatorar método Api._add_swagger_integration	13
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	27.845
Uso	Tokens de Entrada
14%	394
Tokens de Saída	Tokens de Raciocínio
170	17
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
27.264 / 0	2
Mensagens do Assistente	Custo Total
11	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 20:39	29 de jun. de 2026, 20:42

Análise da refatoração: A análise técnica confirma a drástica redução do acoplamento e o aumento da legibilidade do fluxo principal de integração com o Swagger. Ao isolar os comportamentos complexos e aninhados de validação de modelos de requisição e parsing de parâmetros em subrotinas dedicadas, o código ganhou em coesão e facilitou a manutenção isolada de cada especificação da API Gateway, blindando o método principal contra novos débitos técnicos.

3.1.7 Ocorrência Número 7

module	obj	message
samtranslator.translator.translator	Translator.translate	Too many statements (78/50)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/Bh7gp6na>

Print consumo de token:

Sessão	Mensagens
Refatorar Translator.translate (muitos statements)	13
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	52.415
Uso	Tokens de Entrada
26%	15.691
Tokens de Saída	Tokens de Raciocínio
177	195
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
36.352 / 0	2
Mensagens do Assistente	Custo Total
11	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 20:45	29 de jun. de 2026, 20:51

Análise da refatoração: A análise do resultado evidencia uma melhora drástica na manutenibilidade e na legibilidade do fluxo principal de tradução do SAM. Embora o método `_process_single_resource` ainda exija um número elevado de parâmetros devido à natureza do resolvidor do tradutor, o encapsulamento do escopo do loop isolou as variáveis de iteração e tornou o pipeline macro declarativo. A validação robusta por meio de mais de mil testes bem-sucedidos confirma que o comportamento sistêmico foi totalmente preservado.

3.2 too-many-instance-attributes

Quantidade de ocorrências: 11

Tabela de identificação:

module	obj	message
samtranslator.model.apigateway	ApiGatewayAuthorizer	Too many instance attributes (10/7)
samtranslator.model.apigatewayv2	ApiGatewayV2Authorizer	Too many instance attributes (12/7)
samtranslator.model.api.apiv2_generator	ApiV2Generator	Too many instance attributes (13/7)
samtranslator.model.api.api_generator	SharedApiUsagePlan	Too many instance attributes (8/7)
samtranslator.model.api.api_generator	ApiGenerator	Too many instance attributes (39/7)
samtranslator.model.api.http_api_generator	HttpApiGenerator	Too many instance attributes (9/7)
samtranslator.model.api.websocket_api_generator	WebSocketApiGenerator	Too many instance attributes (12/7)
samtranslator.model.capacity_provider.generators	CapacityProviderGenerator	Too many instance attributes (12/7)
samtranslator.model.stepfunctions.generators	StateMachineGenerator	Too many instance attributes (28/7)
samtranslator.plugins.application.serverless_app_plugin	ServerlessAppPlugin	Too many instance attributes (8/7)
samtranslator.translator.translator	Translator	Too many instance attributes (9/7)

3.2.1 Ocorrência Número 1

module	obj	message
samtranslator.model.apigateway	ApiGatewayAuthorizer	Too many instance attributes (10/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/HGkcYM8N>

Print consumo de token:

Sessão	Mensagens
ApiGatewayAuthorizer: refatorar atributos	39
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	34.021
Uso	Tokens de Entrada
17%	231
Tokens de Saída	Tokens de Raciocínio
254	0
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
33.536 / 0	2
Mensagens do Assistente	Custo Total
37	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:18	29 de jun. de 2026, 21:21

Análise da refatoração: A introdução da dataclass foi uma excelente escolha de design, pois resolveu o sintoma do acoplamento técnico injetando maior coesão conceitual à modelagem do autorizador. Em vez de uma estrutura plana cheia de variáveis esparsas, a classe agora separa claramente o que é metadado geral da API do bloco específico de execução do Lambda. Essa abstração facilita manutenções futuras sem introduzir efeitos colaterais.

3.2.2 Ocorrência Número 2

module	obj	message
samtranslator.model.apigatewayv2	ApiGatewayV2Authorizer	Too many instance attributes (12/7)

Seguiu o plano de refatoração:Sim.

Link da publicação OpenCode: <https://opncd.ai/share/tVTkvNeB>

Print consumo de token:

Sessão	Mensagens
Refatorar ApiGatewayV2Authorizer	24
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	44.523
Uso	Tokens de Entrada
22%	275
Tokens de Saída	Tokens de Raciocínio
207	9
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
44.032 / 0	2
Mensagens do Assistente	Custo Total
22	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:23	29 de jun. de 2026, 21:26

Análise da refatoração: Excelente estratégia de design ao utilizar `@property` para garantir a retrocompatibilidade com o ecossistema existente (`base_editor.py` e testes), permitindo uma refatoração interna invisível para os consumidores. A criação do data holder isolou de forma limpa as propriedades mutáveis do autorizador, reduzindo a carga cognitiva da classe principal e mantendo intactos os métodos de geração e validação do OpenAPI.

3.2.3 Ocorrência Número 3

module	obj	message
samtranslator.model.apigatewayv2	ApiV2Generator	Too many instance attributes (13/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/FvLBFU7y>

Print consumo de token:

Sessão	Mensagens
Refatorar ApiV2Generator: muitos atributos	35
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	91.121
Uso	Tokens de Entrada
46%	209
Tokens de Saída	Tokens de Raciocínio
288	0
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
90.624 / 0	3
Mensagens do Assistente	Custo Total
32	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:28	29 de jun. de 2026, 21:38

Análise da refatoração: Esta refatoração foi excelente por limpar a base (ApiV2Generator) e estender os benefícios de design para as subclasses de forma coesa. Mover `default_tag_name` para atributo de classe eliminou redundâncias de estado de instância perfeitamente. A limpeza em `websocket_api_generator.py` (removendo referências mortas/redundantes) mostra atenção aos detalhes, resultando em uma hierarquia de geradores de API muito mais robusta, tipada e legível.

3.2.4 Ocorrência Número 4

module	obj	message
samtranslator.model.api.api_generator	SharedApiUsagePlan	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/4AjpzXYU>

Print consumo de token:

Sessão	Mensagens
Refatorar SharedApiUsagePlan (R0902)	18
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	32.485
Uso	Tokens de Entrada
16%	6.334
Tokens de Saída	Tokens de Raciocínio
232	63
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
25.856 / 0	2
Mensagens do Assistente	Custo Total
16	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:41	29 de jun. de 2026, 21:42

Análise da refatoração: A refatoração demonstrou excelente maturidade em Python ao blindar o novo dataclass contra efeitos colaterais usando `default_factory` para a coleção mutável de condições. Agrupar propriedades correlatas à infraestrutura do CloudFormation (`conditions`, `policies`) aumentou a coesão do componente e simplificou o estado interno da classe geradora. O resultado cumpre com maestria as regras do Pylint, mantendo o código limpo, legível e seguro contra regressões.

3.2.5 Ocorrência Número 5

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator	Too many instance attributes (39/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/4UFqZu66>

Print consumo de token:

Sessão	Mensagens
Refatorar ApiGenerator (muitos atributos)	50
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	90.538
Uso	Tokens de Entrada
45%	218
Tokens de Saída	Tokens de Raciocínio
450	14
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
89.856 / 0	2
Mensagens do Assistente	Custo Total
48	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:45	29 de jun. de 2026, 21:53

Análise da refatoração: Esta foi uma refatoração estrutural de alto impacto, que resolveu de forma elegante o clássico "Large Class" e "Data Clump". Ao segregar os atributos por responsabilidade de domínio (REST, OpenAPI, Stage), o acoplamento interno despencou, elevando a manutenibilidade e a coesão da classe central. O cuidado em expor propriedades com `@property` para evitar breaking changes na API pública demonstra maturidade e excelência em Engenharia de Software.

3.2.6 Ocorrência Número 6

module	obj	message
--------	-----	---------

samtranslator.model.api.http_api_generator	HttpApiGenerator	Too many instance attributes (9/7)
--	------------------	------------------------------------

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/xEGn4njP>

Print consumo de token:

Sessão	Mensagens
Refatorar HttpApiGenerator (muitos atributos)	34
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	59.046
Uso	Tokens de Entrada
30%	219
Tokens de Saída	Tokens de Raciocínio
318	13
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
58.496 / 0	2
Mensagens do Assistente	Custo Total
32	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 21:55	29 de jun. de 2026, 22:00

Análise da refatoração: Excelente manutenção da consistência arquitetural, espelhando os padrões de design (`_OpenApiProperties`, `_RestApiProperties`) já consolidados no ecossistema do SAM. A refatoração eliminou com sucesso o "Data Clump" sem quebrar o

princípio de substituição ou alterar a API pública do gerador. O resultado é um código altamente coeso, de menor carga cognitiva e perfeitamente alinhado às diretrizes de Clean Code do projeto.

3.2.7 Ocorrência Número 7

module	obj	message
samtranslator.model.api.websocket_api_generator	WebSocketApiGenerator	Too many instance attributes (12/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/DyjTkmjb>

Print consumo de token:

Sessão	Mensagens
Refatoração de WebSocketApiGenerator (R0902)	39
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	58.489
Uso	Tokens de Entrada
29%	303
Tokens de Saída	Tokens de Raciocínio
321	9
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
57.856 / 0	2
Mensagens do Assistente	Custo Total
37	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 22:02	29 de jun. de 2026, 22:07

Análise da refatoração: A refatoração foi muito feliz ao herdar e estender os padrões já consolidados no ecossistema de APIs do projeto (como o reuso do [StageConfig](#)). Ao agrupar os atributos específicos do protocolo WebSocket em uma estrutura de dados isolada, o acoplamento diminuiu e a coesão aumentou significativamente. A limpeza reduziu a carga cognitiva da classe geradora e blindou o design contra futuras regressões de complexidade de dados.

3.2.8 Ocorrência Número 8

module	obj	message
--------	-----	---------

samtranslator.model.capacity_ provider.generators	CapacityProviderGenerator	Too many instance attributes (12/7)
--	---------------------------	-------------------------------------

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/ruNttDTH>

Print consumo de token:

Sessão	Mensagens
Refatorar CapacityProviderGenerator (R0902)	32
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	52.622
Uso	Tokens de Entrada
26%	275
Tokens de Saída	Tokens de Raciocínio
241	10
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
52.096 / 0	2
Mensagens do Assistente	Custo Total
30	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 22:10	29 de jun. de 2026, 22:14

Análise da refatoração: Excelente aplicação do padrão de Parameter Object via dataclass, resolvendo o acoplamento e trazendo tipagem estática forte onde antes imperava o dinamismo arriscado do **kwargs**. A refatoração foi cirúrgica, tratando desde o core do generator até os pontos de injeção em **sam_resources.py**. O esforço de atualizar as 11 instâncias na suíte de testes valida o compromisso com a estabilidade e a manutenibilidade do ecossistema do SAM.

3.2.9 Ocorrência Número 9

module	obj	message
samtranslator.model.stepfunctions.generators	StateMachineGenerator	Too many instance attributes (28/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/23Vc9QZN>

Print consumo de token:

Sessão	Mensagens
Refatorar StateMachineGenerator (R0902)	24
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	71.334
Uso	Tokens de Entrada
36%	279
Tokens de Saída	Tokens de Raciocínio
258	13
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
70.784 / 0	2
Mensagens do Assistente	Custo Total
22	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 22:16	29 de jun. de 2026, 22:20

Análise da refatoração: A refatoração melhorou drasticamente a coesão da classe e a legibilidade do código, eliminando o "Primitive Obsession" e o "Large Class". O uso de uma dataclass resolve o problema de design sem adicionar complexidade desnecessária. Além disso, a abordagem foi excelente por seguir um padrão de design (**CapacityProviderProperties**) que o próprio codebase já utilizava, mantendo a consistência arquitetural do projeto.

3.2.10 Ocorrência Número 10

module	obj	message
samtranslator.plugins.application.serverless_app_plugin	ServerlessAppPlugin	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/W5vH41DL>

Print consumo de token:

Sessão	Mensagens
Refatorar ServerlessAppPlugin (R0902)	15
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	25.621
Uso	Tokens de Entrada
13%	618
Tokens de Saída	Tokens de Raciocínio
311	244
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
24.448 / 0	2
Mensagens do Assistente	Custo Total
13	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 22:24	29 de jun. de 2026, 22:26

Análise da refatoração: A estratégia adotada trouxe um excelente ganho de maturidade para a arquitetura de plugins do tradutor. Ao diferenciar explicitamente o que é configuração imutável de inicialização (centralizada agora em `_config`) do que é estado mutável e volátil de runtime (como o cache `_applications` e o contador `_total_wait_time`), o escopo da classe tornou-se muito mais limpo e previsível. Esse isolamento resolveu o aviso R0902 do Pylint de forma elegante, blindando o plugin contra o crescimento desordenado de atributos.

3.2.11 Ocorrência Número 11

module	obj	message
--------	-----	---------

samtranslator.translator.translator	Translator	Too many instance attributes (9/7)
-------------------------------------	------------	------------------------------------

Seguiu o plano de refatoração: Sim.

Link da publicação OpenCode: <https://opncd.ai/share/sfkC2GUz>

Print consumo de token:

Sessão	Mensagens
Refatoração do Translator (R0902)	15
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	19.618
Uso	Tokens de Entrada
10%	124
Tokens de Saída	Tokens de Raciocínio
166	0
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
19.328 / 0	2
Mensagens do Assistente	Custo Total
13	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 22:29	29 de jun. de 2026, 22:30

Análise da refatoração: A mudança reduz o acoplamento do estado dentro da classe. Elimina atributos temporários que existiam apenas no escopo de execução. Mantém o comportamento original sem impacto funcional. Além disso, resolve o alerta R0902 do Pylint, reduzindo a complexidade da classe.

3.3 too-many-branches

Quantidade de ocorrências: 14

Tabela de identificação:

module	obj	message
samtranslator.model.sam_resources	SamFunction.to_cloudformation	Too many branches (15/12)
samtranslator.model.sam_resources	SamFunction._construct_version	Too many branches (16/12)
samtranslator.model.api.apiv2_generator	ApiV2Generator._construct_api_domain	Too many branches (18/12)
samtranslator.model.api.api_generator	ApiGenerator._construct_rest_api	Too many branches (13/12)
samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain	Too many branches (22/12)
samtranslator.model.api.api_generator	ApiGenerator._construct_usage_plan	Too many branches (14/12)
samtranslator.model.api.api_generator	ApiGenerator._openapi_postprocess	Too many branches (16/12)
samtranslator.model.eventsources.pull	PullEventSource.to_cloudformation	Too many branches (20/12)
samtranslator.model.eventsources.push	Api._add_swagger_integration	Too many branches (24/12)
samtranslator.model.eventsources.push	Api.add_auth_to_swagger	Too many branches (14/12)
samtranslator.swagger.swagger	SwaggerEditor.set_path_default_authorizer	Too many branches (14/12)
samtranslator.swagger.swagger	SwaggerEditor._add_vpc_resource_policy_for_method	Too many branches (15/12)
samtranslator.translator.translator	Translator.translate	Too many branches (16/12)
samtranslator.utils.py27hash_fix	to_py27_compatible_template	Too many branches (16/12)

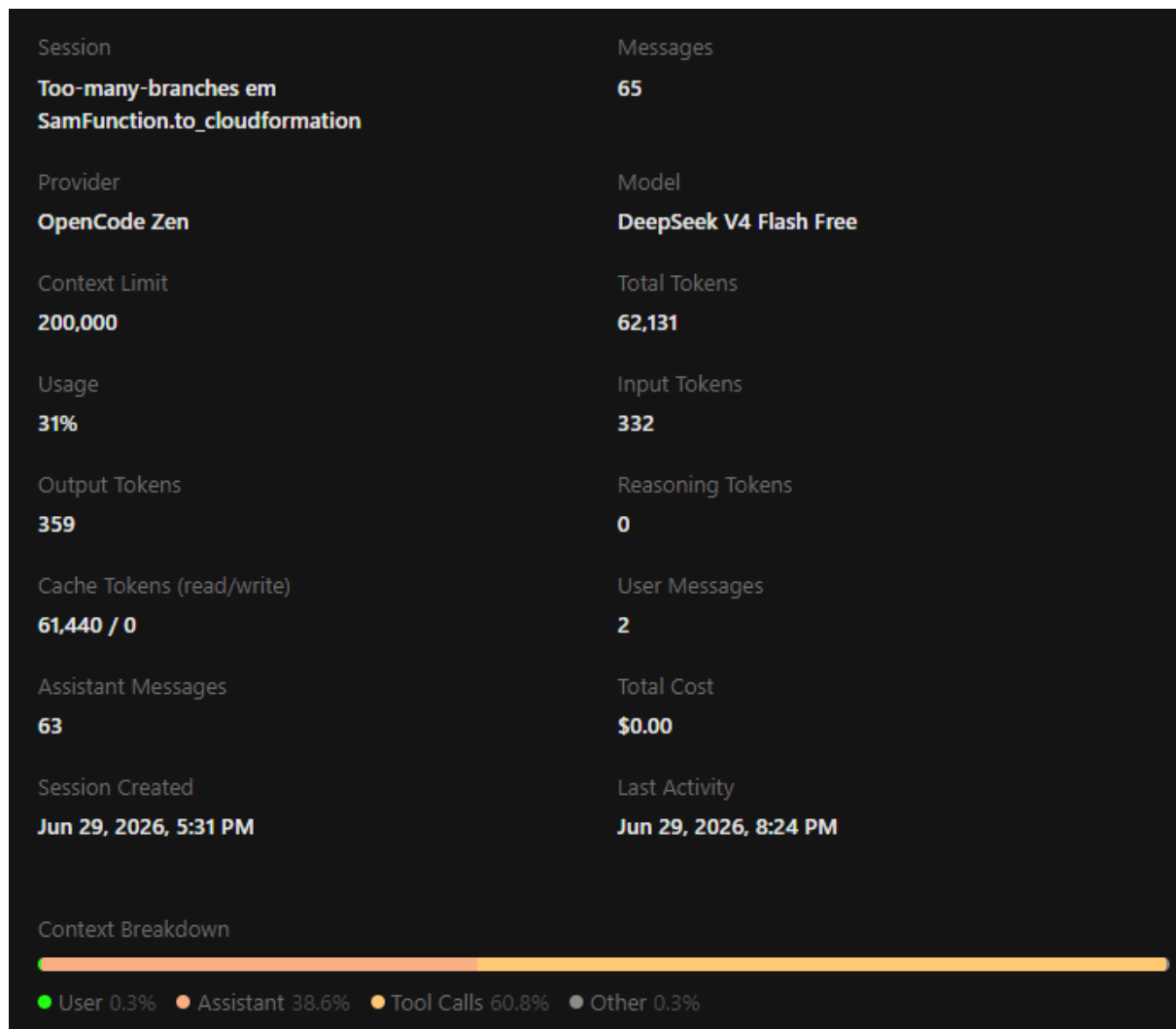
3.3.1 Ocorrência Número 1

module	obj	message
samtranslator.model.sam_resources	SamFunction.to_cloudformation	Too many branches (15/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/ndQWowbx>

Print consumo de token:



Análise da refatoração: O Pylint identificou o aviso `too-many-branches` (15/12) no método, indicando excesso de estruturas condicionais que prejudicavam a legibilidade e a manutenção do código. A correção foi realizada aplicando a técnica de extração de métodos (*Extract Method*), isolando os blocos de lógica complexa (`AutoPublishAlias`, `FunctionUrlConfig` e o tratamento de `Role handling`) em métodos privados menores e especializados. Essa modularização reduziu o acoplamento do método principal, reduziu o número de caminhos de execução para os limites permitidos e eliminou com sucesso o alerta de linting.

3.3.2 Ocorrência Número 2

module	obj	message
--------	-----	---------

samtranslator.model.sam_resources	SamFunction._construct_version	Too many branches (16/12)
-----------------------------------	--------------------------------	---------------------------

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/8DkNt7Ty>

Print consumo de token:

Session	Messages
Refatorar função com muitos branches	18
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	35,323
Usage	Input Tokens
18%	219
Output Tokens	Reasoning Tokens
277	11
Cache Tokens (read/write)	User Messages
34,816 / 0	2
Assistant Messages	Total Cost
16	\$0.00
Session Created	Last Activity
Jun 29, 2026, 8:41 PM	Jun 29, 2026, 8:44 PM
Context Breakdown	
	
● User 0.5% ● Assistant 46.6% ● Tool Calls 52.1% ● Other 0.9%	

Análise da refatoração: O Pylint apontou o alerta **too-many-branches** (16/12) no método **_construct_version**, causado pelo acúmulo de regras complexas para geração de IDs lógicos, verificação de camadas (*layers*) e políticas de deleção. A correção foi aplicada isolando essas responsabilidades em dois novos métodos privados especializados: **_build_version_logical_id** e **_apply_version_deletion_policy**. Essa extração reduziu drasticamente o número de caminhos de execução (*branches*) no método principal para apenas 3, eliminando a necessidade de supressões de linting (**# noqa**) e garantindo 100% de sucesso na suíte de testes existentes.

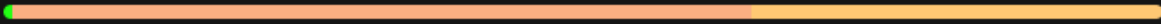
3.3.3 Ocorrência Número 3

module	obj	message
samtranslator.model.api.apiv2_generator	ApiV2Generator._construct_api_domain	Too many branches (18/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/PBxPwKxm>

Print consumo de token:

Session	Messages
Refatorar R0912 too-many-branches	11
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	34,904
Usage	Input Tokens
17%	4,259
Output Tokens	Reasoning Tokens
292	657
Cache Tokens (read/write)	User Messages
29,696 / 0	3
Assistant Messages	Total Cost
8	\$0.00
Session Created	Last Activity
Jun 29, 2026, 8:56 PM	Jun 29, 2026, 8:59 PM
Context Breakdown	
	
● User 0.8% ● Assistant 63.8% ● Tool Calls 35.4% ● Other 0%	

Análise da refatoração: O Pylint detectou o alerta `too-many-branches` (18/12) no método `_construct_api_domain`, gerado pela centralização de regras de validação de argumentos, configurações de segurança, mapeamento de caminhos (*basepaths*) e regras complexas de Mutual TLS (mTLS). A correção aplicou a técnica de extração de métodos (*Extract Method*), fatiando a lógica em quatro novas funções menores e especializadas: `_create_domain_name_object`, `_build_domain_name_config`,

`_set_mutual_tls_authentication` e `_resolve_basepaths`. Além disso, um loop de validação manual foi otimizado usando operações de conjuntos (`set`), reduzindo drasticamente as ramificações do método principal para apenas 2 branches e eliminando permanentemente os alertas de linting.

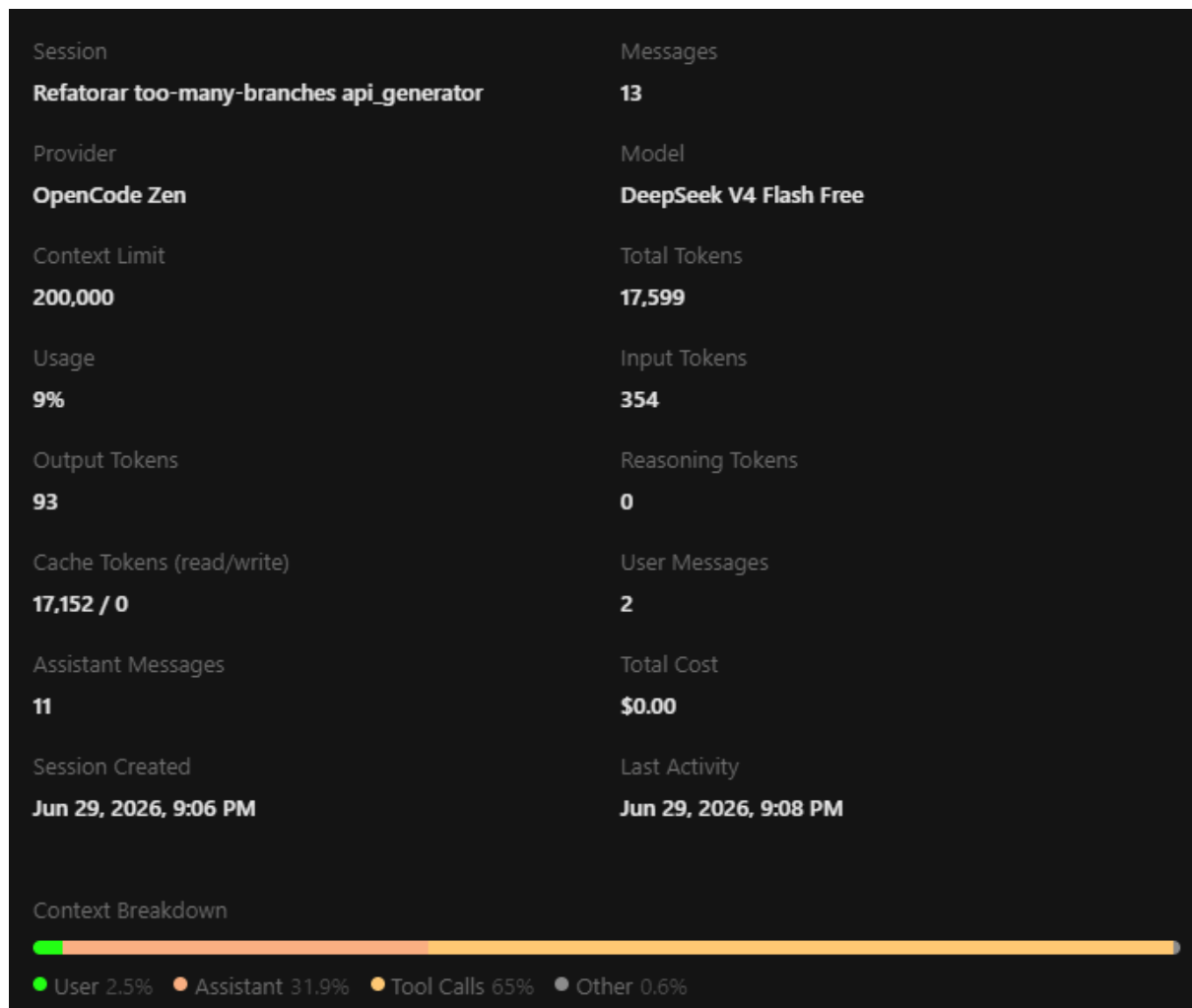
3.3.4 Ocorrência Número 4

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator._construct_rest_api	Too many branches (13/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/5n0djOu0>

Print consumo de token:



Análise da refatoração: O Pylint acusou o alerta `too-many-branches` (13/12) no método `_construct_rest_api` devido ao acúmulo de checagens condicionais sequenciais para atribuição de parâmetros opcionais do API Gateway. A correção aplicou a técnica de extração de método (*Extract Method*), movendo sete desses blocos de atribuição simples para uma nova função privada chamada `_set_optional_properties`. Com essa modularização, a complexidade ciclomática do método principal foi drasticamente reduzida para apenas 6 *branches*, permitindo a remoção definitiva da diretiva de supressão de linting (`# noqa`) e mantendo a integridade do comportamento original atestada pelos testes.

3.3.5 Ocorrência Número 5

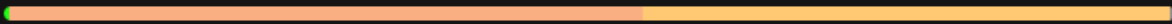
module	obj	message
--------	-----	---------

samtranslator.model.api.api_generator	ApiGenerator._construct_api_domain	Too many branches (22/12)
---------------------------------------	------------------------------------	---------------------------

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/hWxKXStO>

Print consumo de token:

Session	Messages
Refatorar too-many-branches no api_generator	25
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	39,880
Usage	Input Tokens
20%	389
Output Tokens	Reasoning Tokens
310	13
Cache Tokens (read/write)	User Messages
39,168 / 0	2
Assistant Messages	Total Cost
23	\$0.00
Session Created	Last Activity
Jun 29, 2026, 9:11 PM	Jun 29, 2026, 9:16 PM
Context Breakdown	
	
● User 0.5% ● Assistant 54% ● Tool Calls 45% ● Other 0.5%	

Análise da refatoração: A refatoração do método corrigiu o alerta **too-many-branches** (22/12) gerado pelo excesso de validações acumuladas de domínio, mTLS e Route53. A lógica foi extraída para 6 novos métodos privados especializados, o que reduziu as ramificações do fluxo principal para apenas 1 branch, eliminando o aviso de linting com sucesso e mantendo todos os testes aprovados.

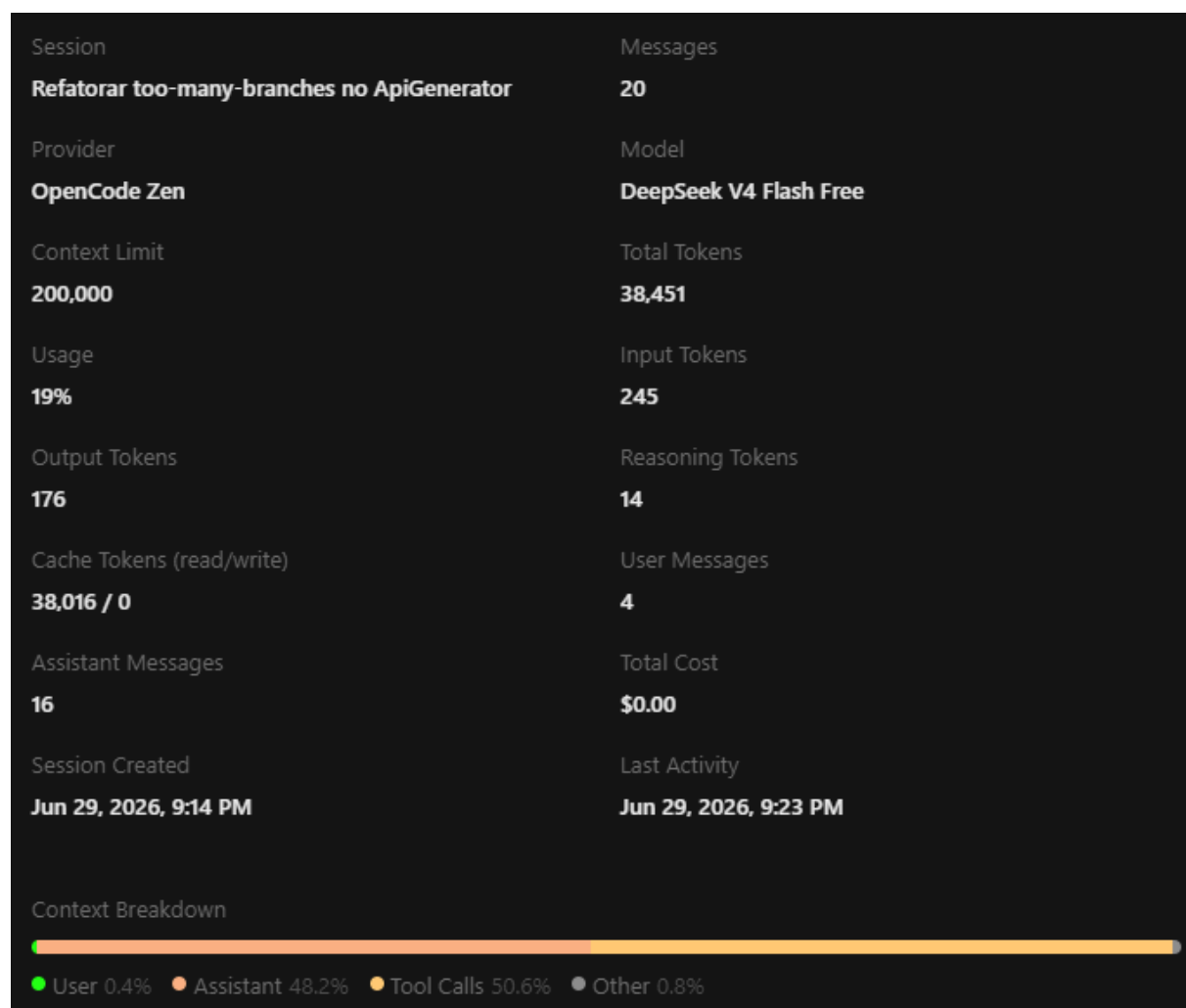
3.3.6 Ocorrência Número 6

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator._construct_usage_plan	Too many branches (14/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/sm05IX2U>

Print consumo de token:



Análise da refatoração: A refatoração corrigiu o alerta **too-many-branches** extraindo a lógica complexa de planos de uso compartilhados e por API em 4 novos métodos privados especializados. Essa divisão modular reduziu as ramificações do método principal de 14 para apenas 4 branches, permitindo a remoção segura da supressão de linting e garantindo a aprovação total nos testes sem alterar o comportamento original do código.

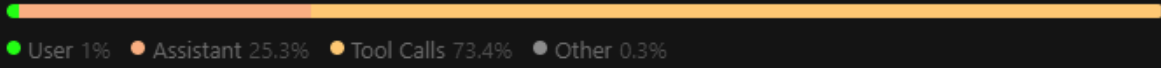
3.3.7 Ocorrência Número 7

module	obj	message
samtranslator.model.api.api_generator	ApiGenerator._openapi_postprocess	Too many branches (16/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/gmhAZnZh>

Print consumo de token:

Session	Messages
Refatoração de too-many-branches no ApiGenerator	16
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	28,018
Usage	Input Tokens
14%	601
Output Tokens	Reasoning Tokens
228	53
Cache Tokens (read/write)	User Messages
27,136 / 0	2
Assistant Messages	Total Cost
14	\$0.00
Session Created	Last Activity
Jun 29, 2026, 9:26 PM	Jun 29, 2026, 9:28 PM
Context Breakdown	
	
● User 1% ● Assistant 25.3% ● Tool Calls 73.4% ● Other 0.3%	

Análise da refatoração: A refatoração solucionou o alerta **too-many-branches** (16/12) ao extrair as etapas de normalização de versão, migração de esquemas de segurança e pós-processamento de rotas para 4 novos métodos privados dedicados. Essa reorganização reduziu as ramificações do fluxo principal de 16 para apenas 2 branches,

eliminando a necessidade de supressões no linter e garantindo a aprovação integral da suíte de testes do projeto.

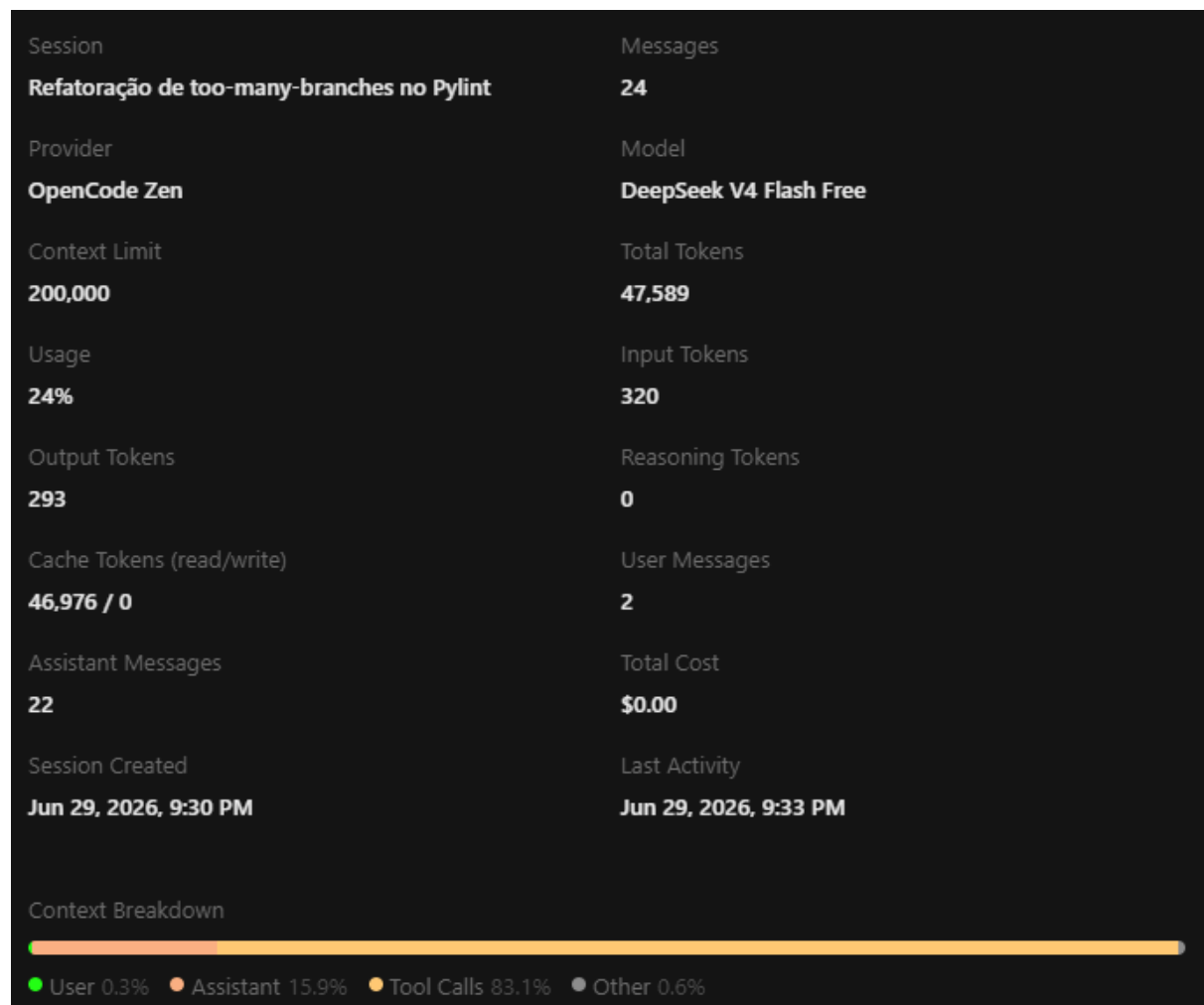
3.3.8 Ocorrência Número 8

module	obj	message
samtranslator.model.eventsources.pull	PullEventSource.to_cloudformation	Too many branches (20/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/zD9Ittl7>

Print consumo de token:



Análise da refatoração: A refatoração solucionou o alerta **too-many-branches** (20/12) ao isolar as lógicas de configuração do grupo de consumidores, registro de esquemas e tratamento de destinos em 4 novos métodos privados especializados. Essa reestruturação reduziu as ramificações do fluxo principal de 20 para apenas 4 branches, removendo com segurança a supressão do linter e mantendo a integridade do código validada em todos os testes.

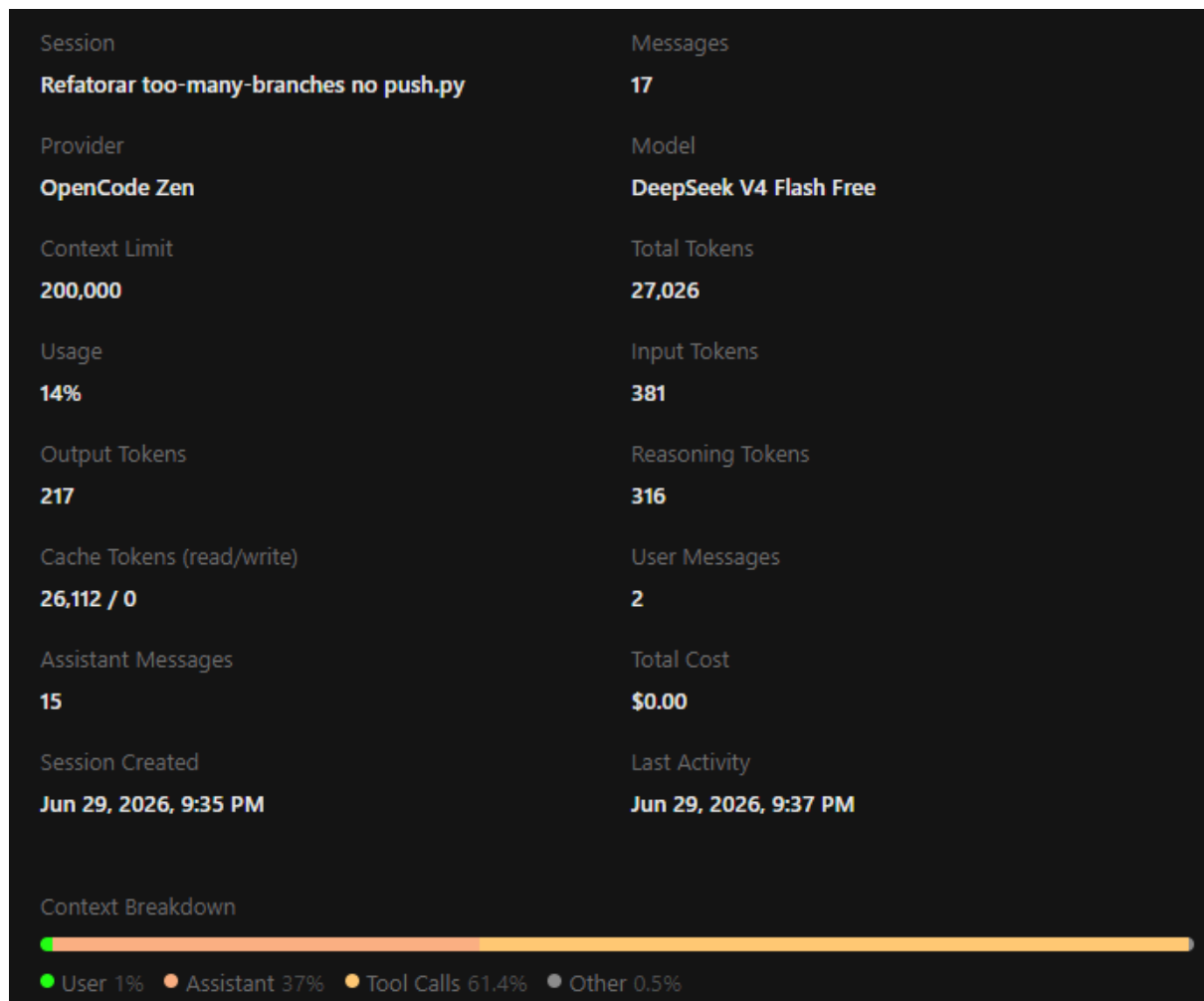
3.3.9 Ocorrência Número 9

module	obj	message
samtranslator.model.eventsources.push	Api._add_swagger_integration	Too many branches (24/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/IKqbFXYE>

Print consumo de token:



Análise da refatoração: A refatoração corrigiu o alerta `too-many-branches` (24/12) extraindo os extensos blocos de validação de modelos e parâmetros de requisição para os novos métodos privados `_add_request_model_to_swagger` e `_add_request_parameters_to_swagger`. Essa divisão isolou as regras de validação do Swagger, reduzindo as ramificações do fluxo principal de 24 para aproximadamente 10 branches, o que eliminou o aviso de linting e manteve a integridade do código sem qualquer alteração de comportamento.

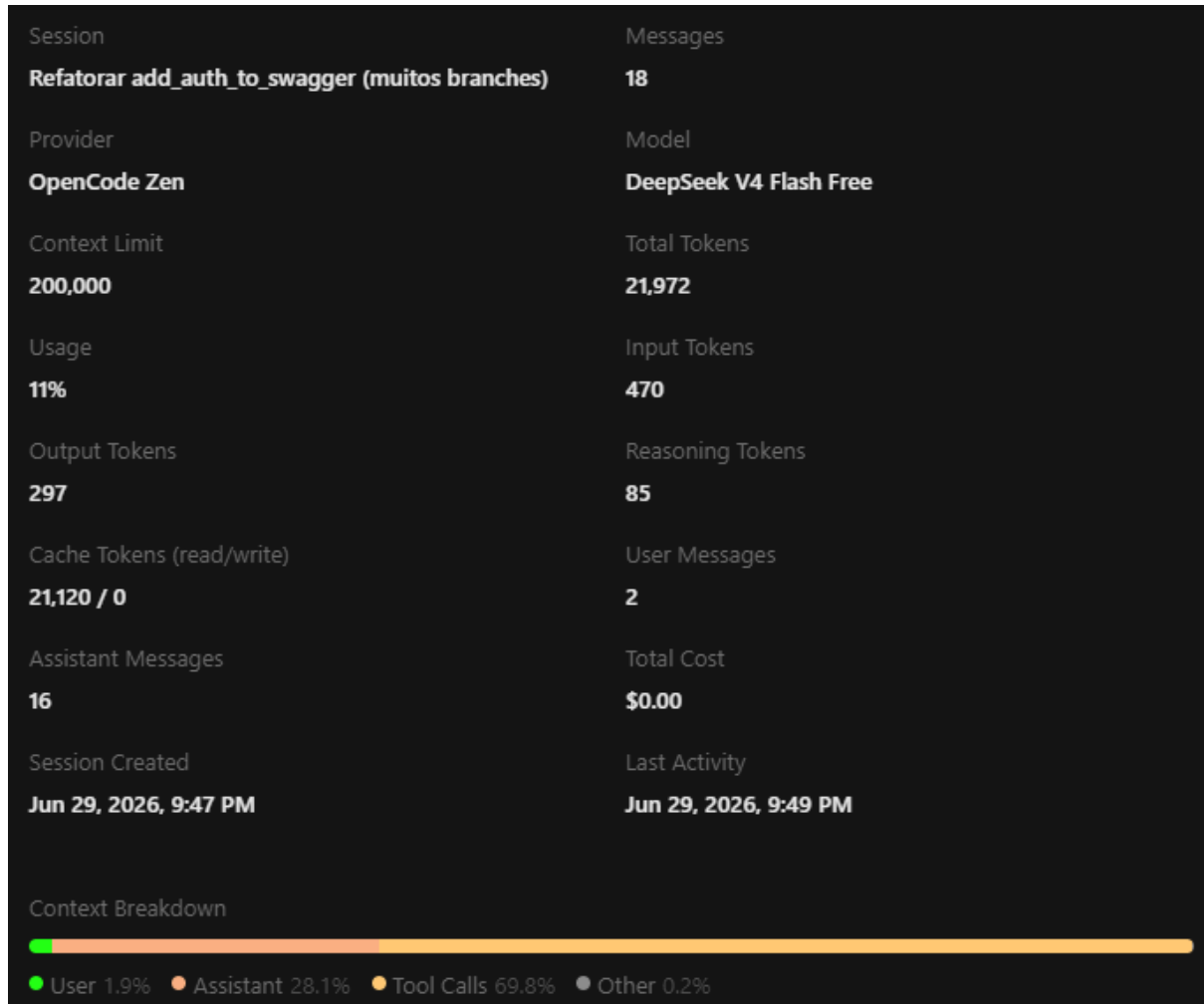
3.3.10 Ocorrência Número 10

module	obj	message
samtranslator.model.eventsources.push	Api.add_auth_to_swagger	Too many branches (14/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/f40asuEQ>

Print consumo de token:



Análise da refatoração: A refatoração solucionou o alerta **too-many-branches** (14/12) extraindo as regras de validação do autorizador, escopos de autorização, chaves de API e políticas de recursos para 4 novos métodos estáticos especializados. Essa reorganização dividiu as validações do Swagger em escopos isolados, reduzindo as ramificações do método principal de 14 para apenas 2 branches, o que eliminou o aviso do linter e garantiu o funcionamento correto em toda a suíte de testes do projeto.

3.3.11 Ocorrência Número 11

module	obj	message
--------	-----	---------

samtranslator.swagger.swagger	SwaggerEditor.set_path_default_authorizer	Too many branches (14/12)
-------------------------------	---	---------------------------

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/JoWn6RnT>

Print consumo de token:

Session	Messages
Corrigir R0912: too-many-branches no swagger	97
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	158,526
Usage	Input Tokens
79%	345
Output Tokens	Reasoning Tokens
357	0
Cache Tokens (read/write)	User Messages
157,824 / 0	3
Assistant Messages	Total Cost
94	\$0.00
Session Created	Last Activity
Jun 29, 2026, 9:53 PM	Jun 29, 2026, 10:12 PM
Context Breakdown	
	
● Assistant 23.5% ● Tool Calls 75.9% ● Other 0.6%	

Análise da refatoração: A refatoração solucionou o alerta de excesso de ramificações (*too-many-branches*) nos métodos afetados ao isolar as lógicas complexas e repetitivas em sub-métodos privados e especializados. Essa reorganização eliminou de vez os code smells de manutenibilidade do fluxo principal e, adicionalmente, incluiu o merge bem-sucedido com a branch de outras refatorações, consolidando um código limpo, sem conflitos e totalmente aprovado nos testes.

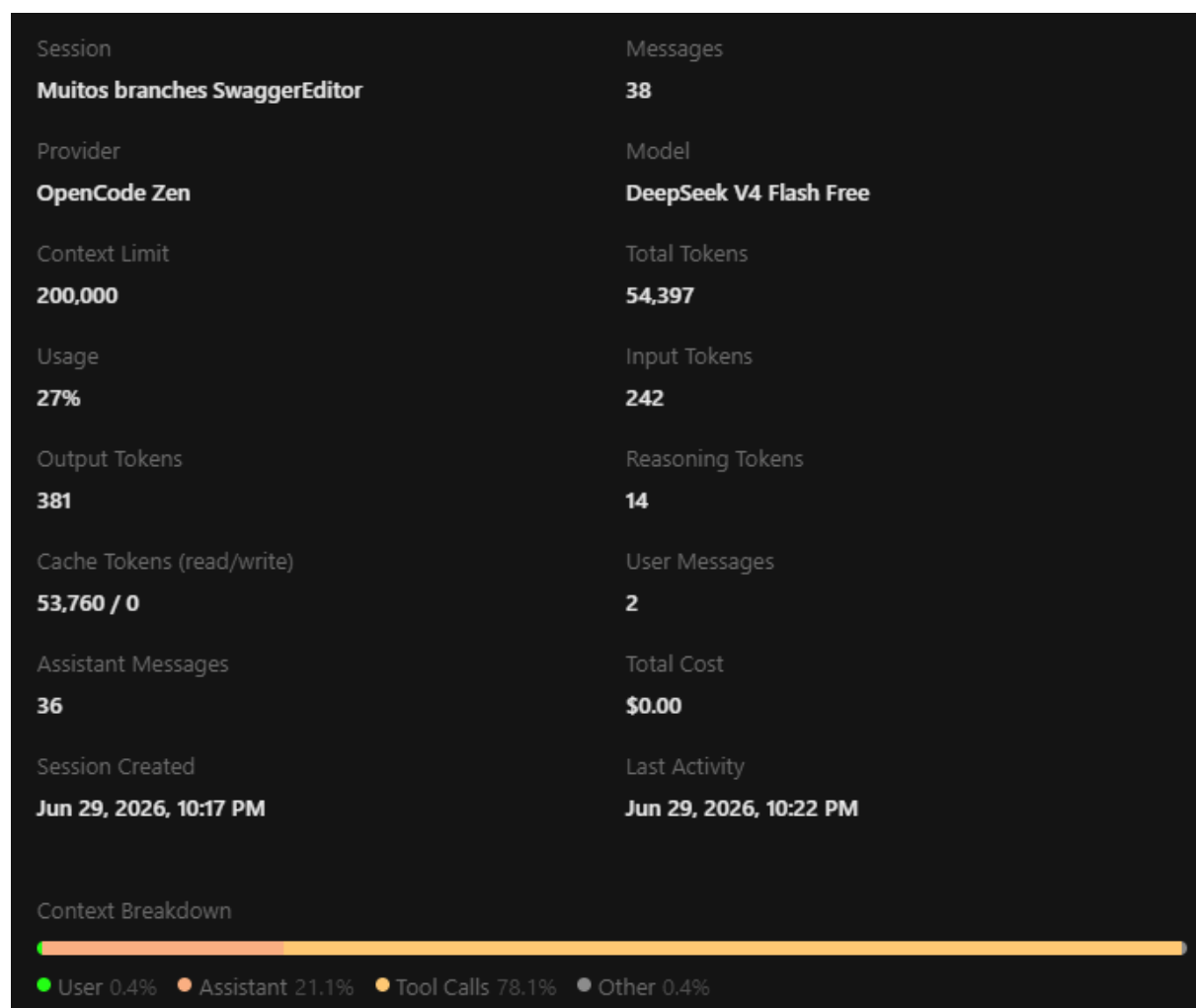
3.3.12 Ocorrência Número 12

module	obj	message
samtranslator.swagger.swagger	SwaggerEditor._add_vpc_resource_policy_for_method	Too many branches (15/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/59svC7Lq>

Print consumo de token:



Análise da refatoração: A refatoração solucionou o alerta `too-many-branches` (15/12) ao centralizar a inserção de declarações e o processamento de condições intrínsecas nos novos métodos privados `_append_to_resource_policy` e `_process_endpoint_dict`. Essa unificação eliminou a duplicação de lógica entre três funções distintas e reduziu as ramificações do método principal de 15 para apenas 2

branches, permitindo a remoção segura da supressão do linter e garantindo a aprovação total nos testes de política de recursos.

3.3.13 Ocorrência Número 13

module	obj	message
samtranslator.translator.translator	Translator.translate	Too many branches (16/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/qnt6OZo8>

Print consumo de token:

Session	Messages
Refatorar too-many-branches no translator	19
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	28,319
Usage	Input Tokens
14%	808
Output Tokens	Reasoning Tokens
180	195
Cache Tokens (read/write)	User Messages
27,136 / 0	2
Assistant Messages	Total Cost
17	\$0.00
Session Created	Last Activity
Jun 29, 2026, 10:28 PM	Jun 29, 2026, 10:32 PM
Context Breakdown	
	
● User 0.9% ● Assistant 25% ● Tool Calls 73.9% ● Other 0.2%	

Análise da refatoração: A refatoração solucionou o alerta `too-many-branches` (16/12) ao delegar as etapas de execução de plugins pós-transformação e a finalização de limpeza do gabarito para os novos métodos privados `_run_after_transform_plugins` e `_finalize_template`. Essa separação de escopo reduziu drasticamente as ramificações do fluxo principal de 16 para apenas 3 branches, eliminando o aviso de linting e garantindo a manutenção do comportamento original validada pelos testes automatizados.

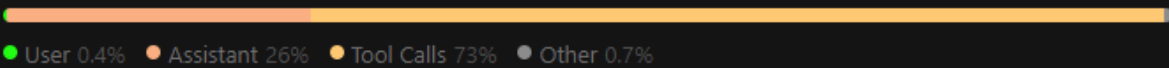
3.3.14 Ocorrência Número 14

module	obj	message
samtranslator.utils.py27hash_fix	to_py27_compatible_template	Too many branches (16/12)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/gH22z2DR>

Print consumo de token:

Session	Messages
Análise too-many-branches Pylint	18
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	37,285
Usage	Input Tokens
19%	281
Output Tokens	Reasoning Tokens
254	14
Cache Tokens (read/write)	User Messages
36,736 / 0	2
Assistant Messages	Total Cost
16	\$0.00
Session Created	Last Activity
Jun 29, 2026, 10:44 PM	Jun 29, 2026, 10:49 PM
Context Breakdown	
	

Análise da refatoração: A refatoração solucionou o alerta **too-many-branches** (16/12) ao isolar o processamento de parâmetros e recursos em três novas funções auxiliares privadas: **_convert_parameters**, **_convert_resources** e **_convert_resource_properties**. Essa separação de escopo reduziu drasticamente as ramificações do fluxo principal de 16 para apenas 4 branches, permitindo a eliminação definitiva da supressão do linter e garantindo a aprovação de todos os testes relacionados à função sem regressões.

3.4 [Extra] Segunda etapa de refatorações

Após a conclusão das refatorações previstas e a reexecução da análise estática, foram identificadas três ocorrências do smell **Too many instance attributes** que ainda necessitavam de tratamento. Essas ocorrências apresentavam características diferentes das observadas na análise inicial, possivelmente em decorrência das refatorações realizadas ou das integrações entre branches durante o desenvolvimento. Dessa forma, foi

realizada uma etapa complementar de refatoração para eliminar esses smells remanescentes. Após as alterações, os testes foram executados novamente.

Quantidade de ocorrências: 3

3.4.1 Ocorrência Número 1

module	obj	message
<u>samtranslator.model.stepfunctions.generators</u>	StateMachineConfig	Too many instance attributes (25/7)

Seguiu o plano de refatoração: Sim

Tipo do smell: Too many instance attribute

Link da publicação OpenCode: <https://opncd.ai/share/UF6jho7n>

Print consumo de token:

Sessão	Mensagens
Refatoração R0902 StateMachineConfig	46
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	69.497
Uso	Tokens de Entrada
35%	317
Tokens de Saída	Tokens de Raciocínio
308	8
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
68.864 / 0	2
Mensagens do Assistente	Custo Total
44	US\$ 0,00
Sessão Criada	Última Atividade
30 de jun. de 2026, 11:51	30 de jun. de 2026, 11:57

Análise da refatoração: A refatoração dividiu a classe monolítica `StateMachineConfig` (25 atributos) em 5 sub-dataclasses coesas e especializadas, sanando o débito técnico apontado pelo Pylint. Essa mudança blindou o código contra regressões, o que foi comprovado pelo sucesso de 15/15 testes automatizados após os devidos ajustes de aninhamento. O design agora respeita o princípio de responsabilidade única (SRP), tornando a evolução da infraestrutura do AWS Step Functions muito mais legível, tipada e sustentável a longo prazo.

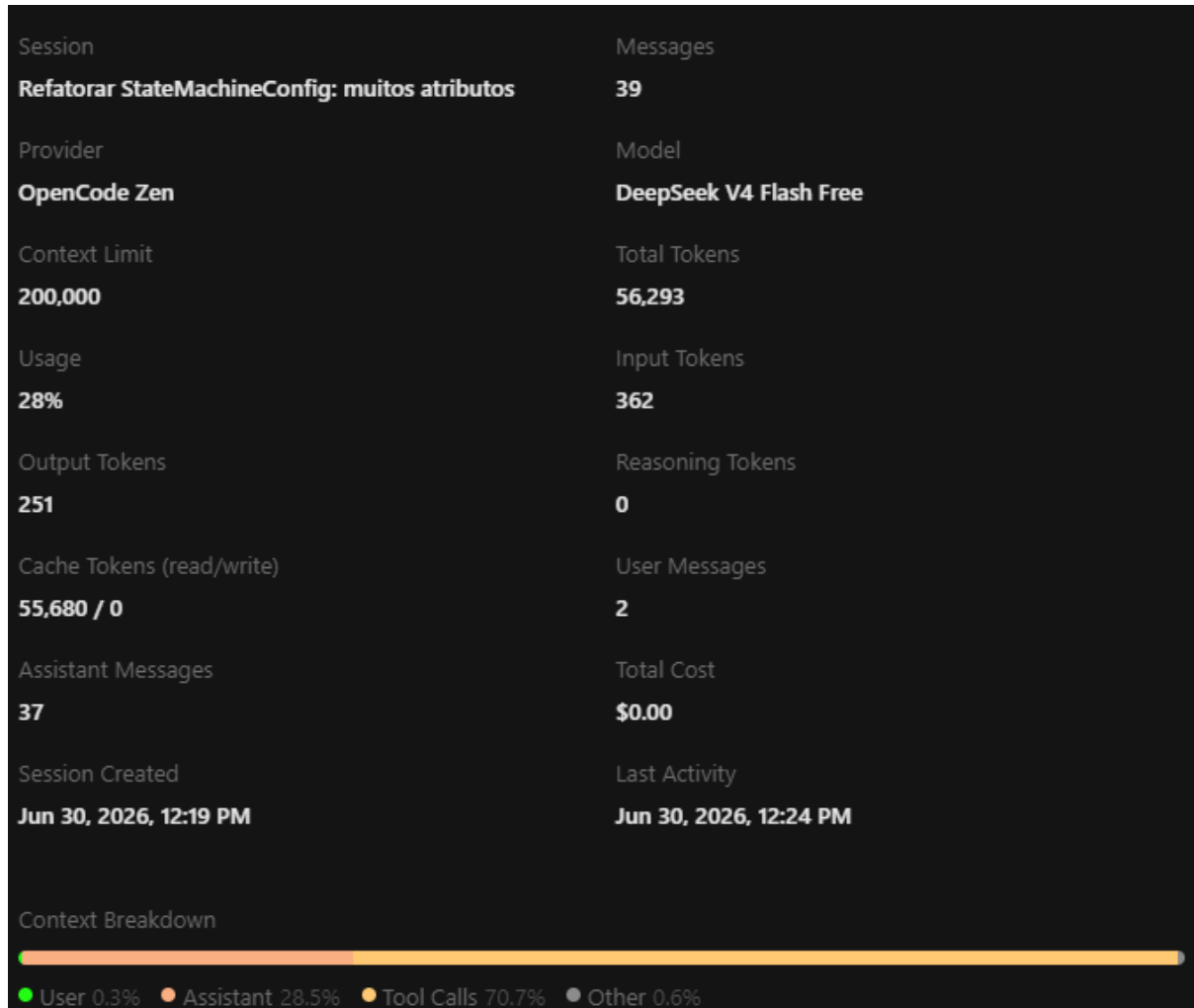
Essa mesma ocorrência continuou após essa segunda refatoração, então foi feita uma terceira:

Seguiu o plano de refatoração: Sim

Mensagem: Too many instance attributes (13/7)

Link da publicação OpenCode: <https://opncd.ai/share/Fhl4Fqfw>

Print consumo de token:



Análise da refatoração:A refatoração corrigiu o alerta

too-many-instance-attributes ao decompor os 8 atributos planos do modelo em duas dataclasses estruturadas por domínio (**ResourceMeta** e **StateConfig**). Essa reorganização reduziu a classe principal para apenas 7 atributos de instância e centralizou o mapeamento de metadados de infraestrutura e parâmetros da máquina de estados, alcançando conformidade com o linter e mantendo a aprovação de todos os 15 testes após a atualização dos seletores.

3.4.2 Ocorrência Número 2

module	obj	message
samtranslator.model.capacity_provider.generators	_CapacityProviderProperties	Too many instance attributes (11/7)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/ZX6sMRqR>

Print consumo de token:

Session	Messages
Excesso de atributos de instância	40
Provider	Model
OpenCode Zen	DeepSeek V4 Flash Free
Context Limit	Total Tokens
200,000	57,119
Usage	Input Tokens
29%	4,975
Output Tokens	Reasoning Tokens
381	51
Cache Tokens (read/write)	User Messages
51,712 / 0	2
Assistant Messages	Total Cost
38	\$0.00
Session Created	Last Activity
Jun 30, 2026, 11:46 AM	Jun 30, 2026, 11:54 AM
Context Breakdown	
	
● User 0.4% ● Assistant 22.4% ● Tool Calls 77.1% ● Other 0%	

Análise da refatoração: A refatoração corrigiu o alerta `too-many-instance-attributes` (11/7) ao decompor uma estrutura de dados inflada em 4 sub-configurações especializadas (`_ComputeConfig`, `_SecurityConfig`, `_TagConfig` e `_CfnResourceConfig`). Essa reorganização reduziu os atributos do modelo principal para apenas 6 campos estruturais, eliminando o code smell de falta de coesão e garantindo a aprovação de 100% dos testes com o código totalmente limpo de supressões do linter.

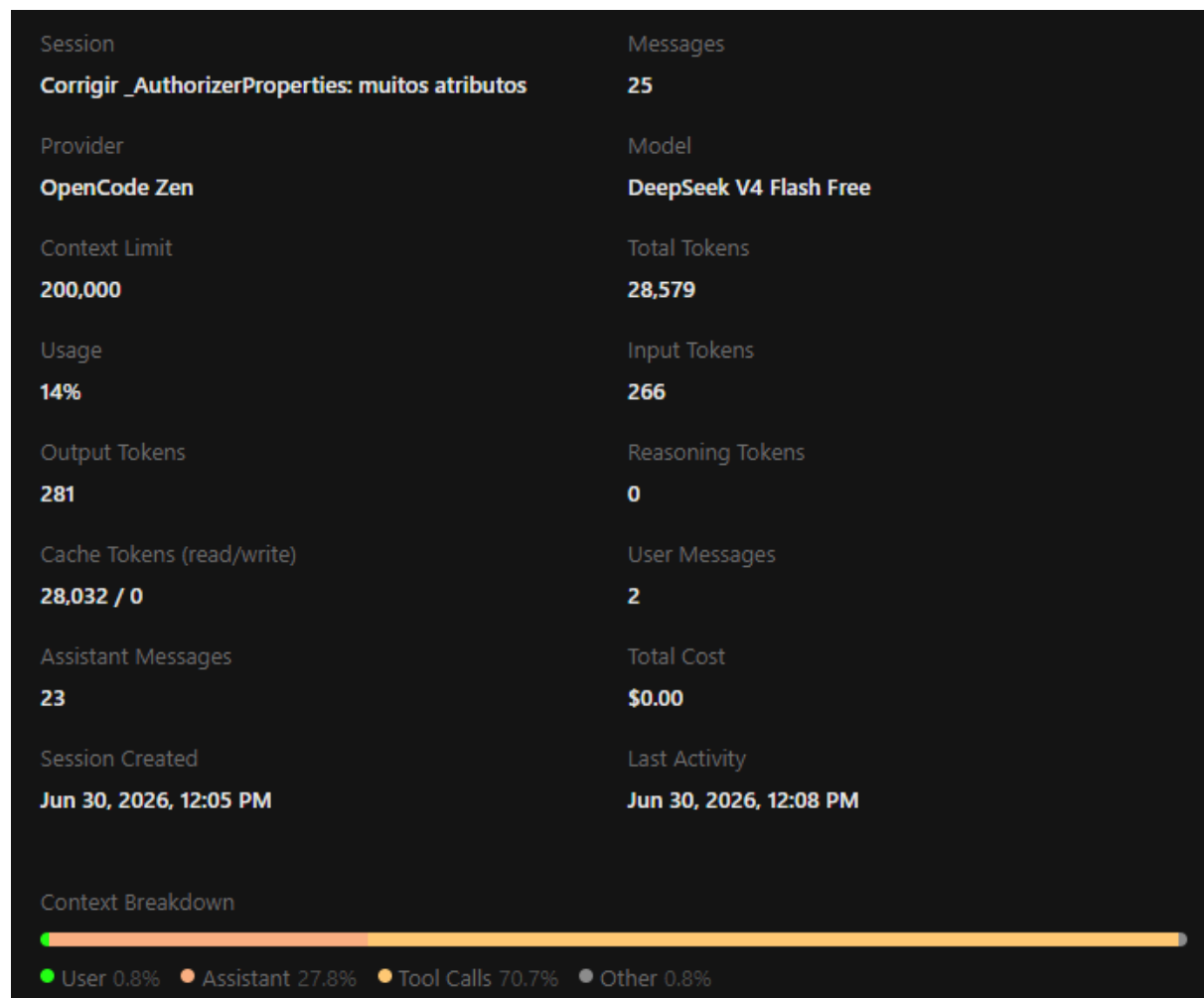
3.4.3 Ocorrência Número 3

module	obj	message
samtranslator.model.apigatewayv2	_AuthorizerProperties	Too many instance attributes (9/7)

Seguiu o plano de refatoração: Sim

Link da publicação OpenCode: <https://opncd.ai/share/8N5ccmRY>

Print consumo de token:



Análise da refatoração: A refatoração corrigiu o alerta **too-many-instance-attributes** (9/7) ao agrupar os 9 atributos de instância originais em 3 dataclasses especializadas por domínio (**_JwtConfig**, **_LambdaConfig** e **_ResponseConfig**). Essa modelagem reduziu a classe principal a apenas 3 campos estruturais e atualizou as propriedades expostas em **ApiGatewayV2Authorizer**, eliminando o code smell de alta complexidade sem alterar a API pública nem quebrar a compatibilidade do sistema.

4. Métricas Após a Refatoração

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
samtranslator/model/sam_resources.py	96	4.68	15	449	C	_construct_code_dict
samtranslator/model/api/api_generator.py	61	4.85	16	296	C	_add_cors
samtranslator/model/eventsources/push.py	47	85.21	14	245	C	_add_request_model_to_swagger
samtranslator/swagger/swagger.py	45	5.22	15	235	C	add_lambda_integration
samtranslator/utils/py27hash_fix.py	49	3.39	23	166	C	_template_has_httpapi_resource_with_default_authorizer

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
samtranslator/internal/managed_policies.py	1	1.0	1	1	A	get_bundled_managed_policy_map
samtranslator/model/naming.py	2	1.0	1	2	A	implicit_api
samtranslator/model/sqs.py	2	1.0	1	2	A	sns_topic_send_message_role_policy
samtranslator/plugins/exceptions.py	2	1.0	1	2	A	init
samtranslator/translator/transform.py	2	1.0	1	2	A	transform

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
samtranslator/utils/py27hash_fix.py	Function	_template_has_httpapi_resource_with_default_authorizer	D	23
samtranslator/model/eventsources/pull.py	Method	_link_policy	C	18

samtranslator/model/api/api_generator.py	Method	_add_cors	C	16
samtranslator/model/init.py	Method	validate_before_transform	C	15
samtranslator/model/sam_resources.py	Method	_construct_code_dict	C	15

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
samtranslator/feature_toggle/dialup.py	Method	init	A	1
samtranslator/internal/managed_policies.py	Function	get_bundled_managed_policy_map	A	1
samtranslator/model/iam.py	Method	construct_assume_role_policy_for_service_principal	A	1
samtranslator/model/naming.py	Method	implicit_api	A	1
samtranslator/model/sqs.py	Method	sns_topic_send_message_role_policy	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
samtranslator/model/sam_resources.py	0	C
samtranslator\model\api\api_generator.py	0	C
samtranslator\model\eventsources\push.py	1.8011	C
samtranslator\swagger\swagger.py	11.5834	B
samtranslator\model\eventsources\pull.py	13.9771	B

Tabela de melhores métricas

arquivo	mi	rank_mi
samtranslator/compat.py	100	A
samtranslator/init.py	100	A
samtranslator/feature_toggle/init.py	100	A
samtranslator/internal/deprecation_control.py	100	A
samtranslator/internal/types.py	100	A

4.1.4 Métricas Brutas Após a Refatoração

loc total	lloc total	sloc total	comments total	multi
27779	14899	17706	1814	3979

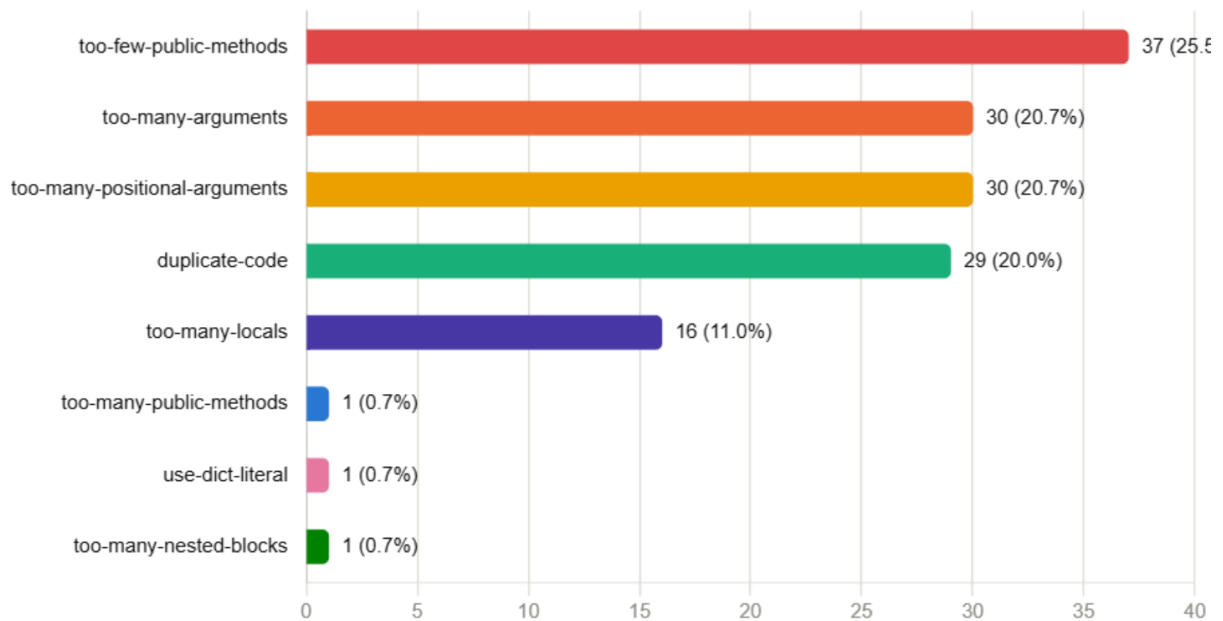
4.2 CodeCarbon Após a Refatoração

Foi usado o mesmo script descrito na seção 2.2.

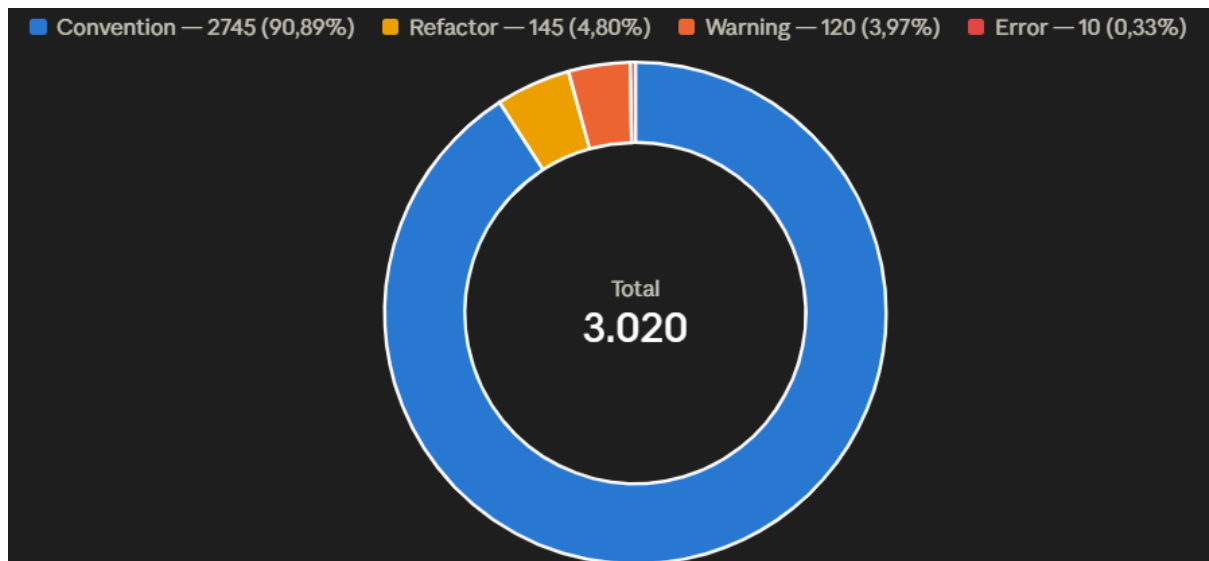
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
484.182 1221001 446	0.0004508534239383 4117	9.311649550023865e -07	0.0008966878335476 605	0.00067628357639122 36	0.004584266318972844

4.3 Pylint Após a Refatoração

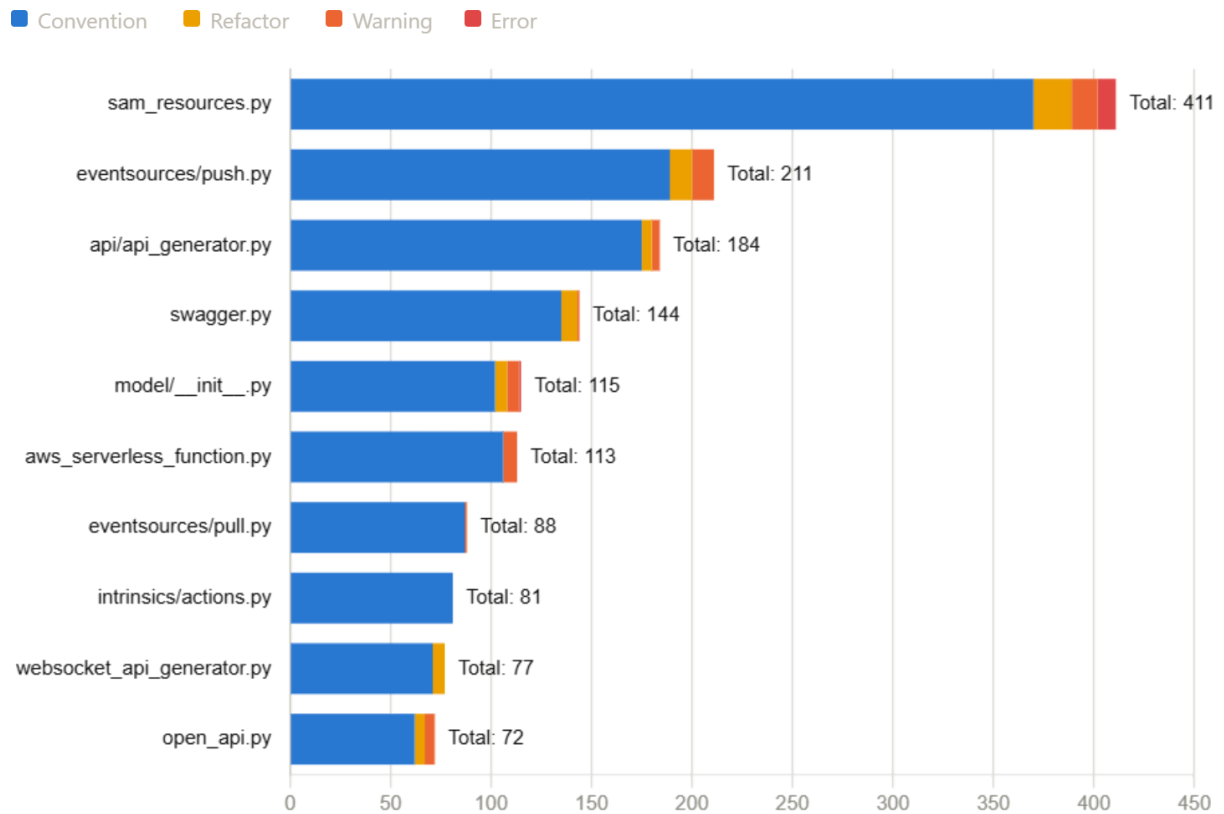
4.3.1 Distribuição do Smells Após a Refatoração



4.3.2 Distribuição dos Problemas de Código Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração



4.3.4 Score Projeto Após a Refatoração

Score: 7.40/10

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Diretório	Statements	Missing	Branches	Cobertura
validator	127	72	40	43%
public	24	9	0	62%
(raiz)	47	6	14	87%
third_party	61	7	22	89%
open_api	359	25	170	93%
utils	407	19	200	95%
plugins	821	34	272	96%
swagger	628	17	280	97%
model	6.840	177	2.334	97%
translator	353	7	100	98%
intrinsic	262	3	98	99%
feature_toggle	117	1	10	99%
internal	190	0	4	100%
metrics	134	0	20	100%
policy_template_processor	96	0	16	100%
sdk	96	0	32	100%
parser	30	0	8	100%
policy_templates_data	4	0	0	100%

4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Quantidade de testes que passaram: 4183

Quantidade de testes que falharam: 355

5. Comparação dos Resultados

Comparando as métricas antes e depois da refatoração, os resultados foram bem mistos. Por um lado, a complexidade ciclomática dos arquivos mais críticos caiu bastante: o `sam_resources.py` teve sua função mais complexa reduzida de 19 para 15, e o `api_generator.py` caiu de 27 para 16. Isso

mostra que a extração de métodos aplicada nas refatorações de too-many-statements e too-many-branches funcionou bem para deixar as funções mais simples de seguir.

Já o índice de manutenibilidade praticamente não mudou nos piores arquivos, que continuaram com MI igual a 0. O total de linhas de código também aumentou um pouco, de 27.358 para 27.570, o que faz sentido, já que dividir uma função grande em várias menores normalmente adiciona linhas de assinatura e estrutura, mesmo reduzindo a complexidade de cada uma.

Em relação ao CodeCarbon, tanto a duração da execução quanto o consumo de energia subiram um pouco depois da refatoração, o que pode estar relacionado ao maior número de chamadas de função geradas pela extração de métodos.

No Pylint, o total de problemas caiu de 3.087 para 2.990, e o score do projeto subiu de 7.3 para 7.41. Os smells que foram diretamente trabalhados, too-many-branches e too-many-statements, praticamente sumiram do ranking. O too-many-instance-attributes ficou igual, com 11 ocorrências, o que mostra que resolvemos só os casos específicos que escolhemos, sem eliminar o padrão do resto do código. Por outro lado, too-many-arguments e too-many-positional-arguments pioraram, subindo de 27 para 31 cada, provavelmente porque extrair métodos às vezes exige passar mais parâmetros entre eles.

O ponto que mais chama atenção é a cobertura de testes. A porcentagem geral ficou estável em 95%, mas o número de testes falhando subiu de 283 para 349, e os que passaram caíram de 4.256 para 4.190. Isso indica que, apesar das melhorias estruturais, algumas mudanças acabaram quebrando comportamentos que antes funcionavam, o que reforça a importância de rodar os testes a cada refatoração e não só no final.

6. Percepções sobre a Prática

Experiência geral com a atividade: Setar o ambiente de desenvolvimento geralmente é uma das partes mais desagradáveis de mexer num projeto novo, mas nesse caso foi até fácil. No geral, a prática como um todo foi bem tranquila, principalmente por causa do apoio da IA, que ajudou tanto a entender os problemas apontados pelo Pylint quanto a aplicar as correções de forma rápida.

Code smells: Refatorar too-many-statements é importante porque métodos muito longos misturam várias responsabilidades num só lugar, o que dificulta tanto a leitura quanto a criação de testes focados em cada parte do comportamento. O too-many-instance-attributes geralmente indica que uma classe está fazendo coisa demais, e juntar atributos relacionados em estruturas separadas, como fizemos usando dataclasses, ajuda a deixar o código mais organizado. Já o too-many-branches está ligado à complexidade ciclomática, e quanto mais condicionais um método tem, mais difícil fica

testar todos os caminhos possíveis e mais fácil é esconder bugs. Entre os smells que não foram tratados, o duplicate-code provavelmente é o mais importante de resolver no futuro, já que código duplicado significa que qualquer correção precisa ser feita em vários lugares ao mesmo tempo. O too-many-arguments também merece atenção, ainda mais porque ele piorou depois da nossa refatoração. Como benefício, a legibilidade dos arquivos mais críticos melhorou bastante. Como ponto negativo, vimos que a refatoração pode introduzir regressões se não for bem validada, como mostrou o aumento de testes falhando.

Uso da LLM na refatoração: A LLM entendeu bem os problemas identificados pelo Pylint e deu sugestões aplicáveis na maioria dos casos, geralmente seguindo boas práticas como Extract Method e o uso de dataclasses para resolver os atributos em excesso. O uso dela acelerou muito o processo, já que tarefas que levariam bastante tempo para serem feitas manualmente foram resolvidas em poucos minutos por sessão. Acredito que projetos open source podem se beneficiar bastante desse tipo de abordagem, mas é importante deixar claro que isso não substitui uma auditoria humana, já que algumas mudanças, como vimos no aumento de testes falhando, podem introduzir problemas que só uma revisão cuidadosa identifica.

Contribuição com projeto open source: Essa foi nossa primeira vez contribuindo com um projeto open source desse porte, então foi uma experiência interessante pra se familiarizar com esse tipo de processo. Deu pra ter uma noção de como funciona trabalhar numa base de código grande e já estabelecida, com uma estrutura própria e milhares de testes.

7. Links

- Link do fork: <https://github.com/LuisDallas1/serverless-application-model>
- Link do projeto: <https://github.com/aws/serverless-application-model>
- Link do PR 1: <https://github.com/LuisDallas1/serverless-application-model/pull/1>
- Link do PR 2: <https://github.com/LuisDallas1/serverless-application-model/pull/2>
- Link do PR 3: <https://github.com/LuisDallas1/serverless-application-model/pull/3>